

**Informe de Resultados de la Suite de Automatización de Pruebas E2E:
SauceDemo con Playwright y TypeScript**

Dangelly Angelica Blandon Negrete

Nuaav — Proceso de Selección

Puesto: QA Automation Engineer

Ejercicio Técnico Offline

23 de junio de 2026

Resumen

El presente informe documenta los resultados de la ejecución de una suite de pruebas automatizadas de extremo a extremo (E2E) desarrollada en TypeScript sobre el framework Playwright, dirigida a la aplicación de práctica SauceDemo. La suite verifica los cinco módulos funcionales de la aplicación —autenticación, catálogo de productos, carrito de compras, proceso de pago y cierre de sesión— y se ejecuta en dos motores de navegador (Chromium y Firefox) bajo un esquema de ejecución totalmente paralela con reutilización de estado de sesión. De un total de 37 pruebas ejecutadas, 36 finalizaron satisfactoriamente y 1 resultó fallida, alcanzando una tasa de aprobación del 97.3%. El análisis de causa raíz determinó que el único fallo no obedece a un defecto de la aplicación ni a un error en la lógica de la prueba, sino a un agotamiento de tiempo de espera (timeout) de naturaleza no determinista, asociado a la contención de recursos del navegador Firefox durante la ejecución paralela en entorno local. Se concluye que los flujos críticos de la aplicación quedan verificados y se formulan recomendaciones para mitigar la intermitencia observada.

Palabras clave: automatización de pruebas, Playwright, TypeScript, pruebas E2E, SauceDemo, control de calidad.

Informe de Resultados de la Suite de Automatización de Pruebas E2E: SauceDemo con Playwright y TypeScript

La verificación automatizada de aplicaciones web constituye una práctica central en la entrega de software de calidad. Este informe presenta, analiza e interpreta los resultados obtenidos tras ejecutar la suite de automatización construida como parte del ejercicio técnico para el puesto de QA Automation Engineer en Nuaav. El objetivo del ejercicio consistió en diseñar una suite mantenible que ejercitara los módulos funcionales de SauceDemo aplicando patrones de diseño reconocidos —en particular el Modelo de Objetos de Página (Page Object Model)— junto con mecanismos de aislamiento de pruebas, fixtures personalizados y persistencia de estado de autenticación. Las secciones siguientes describen la metodología de ejecución, los resultados cuantitativos globales, el desglose por módulo y por navegador, el análisis detallado del fallo registrado, una discusión interpretativa y las conclusiones con sus recomendaciones asociadas.

Metodología de Ejecución

La suite se ejecutó mediante el comando `npx playwright test` desde la raíz del repositorio. El entorno de ejecución empleó Playwright Test como orquestador, con cuatro procesos de trabajo (workers) en paralelo y la opción de ejecución totalmente paralela habilitada (`fullyParallel: true`). La configuración define tres proyectos: un proyecto de preparación (setup) responsable de autenticar una única vez al usuario `standard_user` y persistir su sesión mediante `storageState`, y dos proyectos de navegador —Chromium y Firefox— que dependen del primero y reutilizan dicha sesión. Esta arquitectura evita repetir el proceso de inicio de sesión en cada prueba, reduciendo el tiempo total de ejecución y el acoplamiento entre casos.

La política de tiempos de espera estableció un límite de 45 segundos por prueba, un margen de 10 segundos para las aserciones y presupuestos diferenciados de acción (15 s) y navegación (20 s). La captura de pantalla se configuró exclusivamente ante fallos (`screenshot: 'only-on-failure'`) y la traza de depuración se activó únicamente en el primer reintento. La

generación de reportes se realizó mediante el reporter HTML integrado de Playwright. Conviene destacar que la ejecución analizada se realizó en entorno local, donde la política de reintentos (`retries: 1`) se encuentra deshabilitada por diseño y solo se activa en integración continua mediante la verificación de la variable de entorno `CI`; por consiguiente, cualquier intermitencia se manifiesta sin enmascaramiento.

Resultados Globales

La ejecución completa comprendió 37 pruebas y tuvo una duración total de 4.3 minutos. La Tabla 1 sintetiza los indicadores agregados de la corrida. La tasa de aprobación resultante, del 97.3%, se obtiene de la relación entre las 36 pruebas aprobadas y el total de 37 pruebas ejecutadas.

Tabla 1

Indicadores agregados de la ejecución de la suite

Indicador	Valor
Pruebas totales ejecutadas	37
Pruebas aprobadas	36
Pruebas fallidas	1
Pruebas intermitentes (flaky)	0
Pruebas omitidas (skipped)	0
Tasa de aprobación	97.3%
Duración total	4.3 min
Procesos de trabajo (workers)	4
Navegadores evaluados	2

Nota. Datos obtenidos del reporte HTML de Playwright correspondiente a la ejecución del 23 de junio de 2026, 09:26. La tasa de aprobación se calcula como $36/37 \times 100$.

Desglose por Archivo de Pruebas y Módulo Funcional

La suite se organiza en seis archivos, cada uno alineado con un área funcional de la aplicación. La Tabla 2 presenta la distribución de casos por archivo, indicando los resultados obtenidos. El conteo refleja que la mayoría de los casos se ejecutan en los dos navegadores, de

modo que un mismo escenario contribuye con dos pruebas al total; las excepciones son el proyecto de preparación y los casos de `edge-cases.spec.ts`, que se contabilizan una sola vez por estar vinculados al comportamiento de cuentas específicas.

Tabla 2

Resultados por archivo de pruebas y área funcional cubierta

Archivo	Área funcional	Casos	Aprob.	Fall.
auth.setup.ts	Autenticación y persistencia de sesión	1	1	0
auth.spec.ts	Inicio de sesión, errores y cierre de sesión	10	10	0
inventory.spec.ts	Catálogo y ordenación de productos	10	10	0
cart.spec.ts	Carrito de compras y contador	8	7	1
checkout.spec.ts	Proceso de pago y validación	4	4	0
edge-cases.spec.ts	Cuentas especiales (datos y rendimiento)	4	4	0
Total		37	36	1

Nota. El archivo `auth.spec.ts` ejerce cinco escenarios en dos navegadores (10 pruebas). Los cuatro casos de `edge-cases.spec.ts` incluyen dos escenarios ejecutados en ambos navegadores más comportamientos de cuentas específicas contabilizados de forma única.

Cobertura Funcional Detallada

La Tabla 3 enumera los escenarios concretos verificados por cada módulo de la aplicación. La cobertura abarca tanto trayectorias satisfactorias (happy paths) como estados negativos y de error, así como casos límite asociados a cuentas de comportamiento atípico provistas por SauceDemo.

Tabla 3*Escenarios verificados por módulo funcional*

Módulo	Escenarios verificados	Estado
Autenticación	Inicio exitoso con standard_user; bloqueo de locked_out_user; credenciales inválidas; campo de usuario vacío; cierre de sesión	Aprobado
Catálogo	Renderizado de los seis productos; orden alfabético ascendente y descendente; orden por precio ascendente y descendente	Aprobado
Carrito	Adición de uno y varios artículos; eliminación; actualización del contador; listado en la página del carrito	Aprobado ^a
Pago (checkout)	Flujo completo de tres pasos hasta la confirmación; validación de campo obligatorio ausente	Aprobado
Cuentas especiales	Detección de imágenes defectuosas con problem_user; carga dentro del presupuesto de tiempo con performance_glitch_user	Aprobado

Nota. La cobertura funcional del carrito se verificó íntegramente en ambos navegadores. ^a Un caso registró un agotamiento de tiempo no determinista en Firefox; véase el análisis del fallo.

Análisis Comparativo del Rendimiento por Navegador

Un hallazgo relevante de esta ejecución es la marcada disparidad de tiempos entre los dos navegadores. La Tabla 4 contrasta la duración de un conjunto representativo de casos ejecutados en Chromium y Firefox. De manera sistemática, Firefox presentó tiempos sustancialmente superiores —en varios casos entre tres y cinco veces mayores— lo que constituye el factor determinante del incidente analizado en la sección siguiente.

Tabla 4*Comparación de duración por caso entre Chromium y Firefox*

Caso de prueba	Chromium (s)	Firefox (s)
standard_user inicia sesión correctamente	10.1	36.7
locked_out_user es bloqueado	9.8	40.1
credenciales inválidas muestran error	10.0	44.2
usuario vacío es rechazado	9.9	46.1
el usuario puede cerrar sesión	6.6	24.3
añadir un artículo actualiza el contador	6.0	31.0
añadir varios artículos incrementa el contador	6.0	28.8
eliminar un artículo reduce el contador	6.6	26.0
la página del carrito lista los artículos	7.4	45.7 ^a
checkout completo (happy path)	8.9	23.7
checkout bloquea sin datos del cliente	8.0	22.2

Nota. Duraciones expresadas en segundos. ^a Caso fallido por agotamiento del límite de 45 s durante el hook de preparación; constituye el único incidente de la ejecución.

Análisis del Fallo

El único fallo de la ejecución correspondió al caso *Cart > cart page lists the added items*, ejecutado en Firefox y ubicado en `cart.spec.ts:33`. La Tabla 5 resume sus datos de identificación. El mensaje de error reportado fue el agotamiento del tiempo de espera de 45 000 ms durante la ejecución del hook `beforeEach`, y no en el cuerpo de la prueba.

Tabla 5*Datos de identificación del fallo*

Atributo	Detalle
Caso	Cart › cart page lists the added items
Ubicación	cart.spec.ts:33
Navegador	Firefox
Duración registrada	45.7 s (límite agotado)
Punto de fallo	Hook beforeEach (navegación al inventario)
Mensaje	Test timeout of 45000ms exceeded while running "beforeEach" hook

Nota. Información extraída del contexto de error generado por Playwright para la prueba fallida.

Causa raíz

El timeout se produjo en el hook `beforeEach`, encargado de navegar a la página de inventario y esperar la visibilidad del título "Products" antes de ejecutar el cuerpo de la prueba. Esta circunstancia es determinante para la interpretación: el fallo no se localiza en las aserciones del escenario —que validan el listado de artículos en el carrito— sino en la fase de preparación previa. El mismo caso se ejecutó con éxito en Chromium en 7.4 segundos, lo que descarta un defecto funcional de la aplicación o un error de lógica en la prueba. Como evidencia el análisis comparativo de la Tabla 4, Firefox operó de forma consistentemente más lenta a lo largo de toda la suite, con tiempos que oscilaron entre 22 y 46 segundos frente al rango de 4 a 12 segundos de Chromium. Bajo la configuración de ejecución totalmente paralela con cuatro workers, una instancia concreta de carga en Firefox superó el presupuesto de 45 segundos debido a la contención de recursos del sistema durante la ejecución concurrente. Se trata, por tanto, de una intermitencia (flakiness) de origen ambiental y de rendimiento, no de un defecto reproducible de manera determinista.

Mitigación prevista en la configuración

La configuración de la suite contempla un reintento automático (`retries: 1`) activado exclusivamente en entornos de integración continua mediante la verificación de la variable `CI`.

Dado que la ejecución analizada se realizó en entorno local sin reintentos, la intermitencia se manifestó de forma visible en el reporte. En un entorno de integración continua, el reintento absorbería este caso y la suite reportaría un resultado íntegramente satisfactorio. Esta decisión de diseño es deliberada: mantener los reintentos desactivados en local permite que las intermitencias afloren durante el desarrollo, en lugar de quedar ocultas.

Recomendaciones

A partir del análisis precedente se proponen las siguientes acciones de mejora:

1. Establecer presupuestos de tiempo de espera diferenciados por proyecto, asignando a Firefox un límite superior al de Chromium que refleje su menor velocidad observada en este entorno.
2. Reducir la contención de recursos mediante la limitación del número de workers asignados a Firefox, o habilitar reintentos también en local para los casos sensibles al rendimiento.
3. Confirmar la estabilidad del caso afectado ejecutándolo de forma aislada con el comando `npx playwright test cart.spec.ts -g "cart page lists" --project=firefox`.
4. Considerar la incorporación de esperas explícitas sobre condiciones de red inactiva en el hook de preparación, a fin de robustecer la navegación inicial frente a latencias variables.

Discusión

Los resultados obtenidos permiten afirmar que la suite cumple su propósito de verificar los flujos críticos de SauceDemo con un alto grado de fiabilidad. La cobertura abarca la totalidad de los módulos funcionales requeridos y contempla escenarios tanto positivos como negativos, incluidos los comportamientos atípicos de las cuentas especiales. El único incidente registrado ilustra una distinción metodológicamente importante en el control de calidad: la diferencia entre un fallo funcional —indicativo de un defecto en el producto— y una intermitencia de origen

ambiental, atribuible a las condiciones de ejecución. La identificación correcta de esta distinción evita la atribución errónea de defectos a la aplicación y orienta las acciones correctivas hacia la infraestructura de pruebas. La disparidad de rendimiento entre navegadores, documentada en la Tabla 4, no constituye un defecto en sí misma, pero sí un factor de riesgo que debe gestionarse mediante una configuración de tiempos sensible a las características de cada motor.

Conclusiones

La ejecución de la suite de automatización alcanzó una tasa de aprobación del 97.3%, con 36 de 37 pruebas satisfactorias. La cobertura verificó los cinco módulos funcionales de SauceDemo —autenticación, catálogo, carrito, pago y cierre de sesión— en los navegadores Chromium y Firefox. El único fallo se atribuyó a un agotamiento de tiempo de espera de naturaleza no determinista, originado por la contención de recursos de Firefox durante la ejecución paralela en entorno local, y se encuentra cubierto por la política de reintentos prevista para integración continua. En consecuencia, se considera que la calidad funcional de la aplicación bajo prueba queda verificada en todos sus flujos críticos, y que la suite constituye una base sólida y mantenible para su integración en un proceso de entrega continua, sujeta a las recomendaciones de afinamiento de rendimiento formuladas en este informe.